Segurança Informática e nas Comunicações (SIC)

Sistemas Operativos (Aula 11)

Identidade (Linux) [1/3]

Os utilizadores têm um *id* numérico que os identifica nas operações internas do SO. Associado a esse identificador há uma string com o nome do utilizador, que é usada em substituição desse *id (i.e., geralmente dá-se o nome e depois o sistema resolve qual o id).*

O comando *id* sem parâmetros permite verificar qual o utilizador atual.
(uid == user id)

Pex.,
user@vm:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),4(adm), 27(sudo), … ,999(vboxsf)

Identidade (Linux) [2/3]

Pex.,
user@vm:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),4(adm), 27(sudo), … ,999(vboxsf)

Como podemos verificar, um utilizador tem na sua identidade mais características do que somente o seu **uid**. Cada utilizador tem um grupo primário (**gid**) que o caracteriza (i.e., está armazenado juntamente com a definição desse utilizador). Adicionalmente, o utilizador pode pertencer a mais grupos (**groups**), sendo essa associação feita separadamente da definição desse utilizador.

Identidade (Linux) [3/3]

De forma muito prática,

“definição desse utilizador” == linha no **/etc/passwd**

“associação de grupos” == adicionar nome do utilizador na linha que define o grupo em **/etc/group**

Superutilizador (i.e., root)

Em Linux há um utilizador especial para o qual *(quase*)* tudo é permitido, e é identificado sempre pelo id == 0 (root).

(*) – a exceção são os controlos do tipo MAC, como pex., SELinux ou AppArmor *[vamos ver mais à frente]*.

Permissões

Em Linux tudo o que é controlável é representado num nó na árvore de sistema.

As permissões são definidas diretamente no nó correspondente e em octal *(i.e., [0-7])*, numa conversão direta em BCD do valor dos bits r-w-x.

r – leitura

w – escrita

x – exploração / execução

As permissões são descriminadas para o dono desse nó (utilizador e grupo), e também se define permissões para os restantes (i.e., não donos desse nó).

O comando *chmod* permite (re)definir as permissões dos nós de sistema.

Permissões

Exemplo,

-rwxrwx--- 1 user user 1023 jan 17 15:35 Ex1.py

Octal 770:

o utilizador e grupo “user” podem fazer todas as operações sobre o ficheiro, mas todos os restantes não conseguem fazer-lhe nada.

Como obter root?

- Logando-se com esse utilizador de forma direta, pex., durante o login no sistema ou usando o comando *su*.
  - A identidade do utilizador é o root, obriga a saber as credencias desse utilizador *(i.e., root)*, e ficasse logado no sistema como tal.

- Durante um contexto de execução, usando o comando *sudo*.
  - Continua-se logado como um utilizador que invoca aquele comando, mas naquele contexto de execução torna-se *efetivamente* o utilizador root.

uid == identificador do utilizador

euid == identificador de utilizador efetivo nesse contexto de execução

Como funciona o utilizador efetivo?

Caso de uso: o binário que implementa o comando *sudo*

(/usr/bin/sudo)

Como pode o comando *sudo* funcionar se o utilizador que invoca esse binário não é o root, não há um *deamon sudo* a correr como root, nem é necessário que o user root esteja logado no sistema?

Como funciona o utilizador efetivo?

Caso de uso: comando *sudo*

```
user@vm:~$ ls -la /usr/bin/sudo
-rwsr-xr-x 1 root root 166056 jan 19  2021 /usr/bin/sudo
user@vm:~$
```

-rwsr-xr-x 1 root root 166056 jan 19 2021 /usr/bin/sudo

Há um conjunto de bits adicionais para descrever permissões especiais. A permissão de “Set UID”, que faz com que o contexto de execução passe a ser o do **dono desse nó**. Como o dono desse nó é o *root*, o binário sudo executa-se efetivamente como se fosse invocado pelo user root.

*(**nota**: se o dono fosse pex., user2, era executado como se fosse invocado pelo user2)*

Como funciona o utilizador efetivo?

Caso de uso: comando *sudo*

```
user@vm:~$ ls -la /usr/bin/sudo
-rwsr-xr-x 1 root root 166056 jan 19  2021 /usr/bin/sudo
user@vm:~$
```

Neste caso, o controlo de acesso às permissões de root é feito na lógica do executável *sudo*.

(i.e., um outro comando sem controlo de acesso daria permissão de root *sem pedir mais nada*)

Sou *root (id == 0)* ⇒ consigo fazer tudo?

Geralmente sim, mas… há mecanismos de confinamento que restringem as capacidades dentro daquele contexto de execução, independentemente do utilizador.

<u>Nesta aula vamos começar por:</u>

1) Redefinir uma árvore de sistema para um contexto de execução *(e.g., chroot)*

2) AppArmor *(ou SELinux)*

Redefinir a árvore de sistema [1/3]

Em Linux tudo o que é controlável está exposto num nó da árvore de sistema.

Logo, se naquele contexto de execução reduzirmos os nós expostos aos estritamente necessários, deixa-se de poder fazer ações sobre outros nós que não fazer parte daquela execução (*).

Pex., não se consegue aceder a um ficheiro com um segredo se ele não estiver nessa árvore (*).

(*) – na prática há técnicas que permitem escapar ao chroot

Redefinir a árvore de sistema [2/3]

Pex., se o programa *"server"* apenas necessitar destes nós (i.e., ficheiros):

linux-vdso.so.1 (0x00007ffc32f2f000)

libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f69b01d9000)

/lib64/ld-linux-x86-64.so.2 (0x00007f69b07cd000)

*Ao se criar uma **árvore aparente** que apenas contém o executável server mais os nós listados acima (i.e., suas dependências), deixa-se de conseguir aceder/operar sobre qualquer outro nó fora desta árvore.*

Redefinir a árvore de sistema [3/3]

Problema: O mecanismo de definição de uma árvore de sistema aparente (chroot) não foi concebido para ser um controlo de segurança *(por si só)*, havendo inclusive técnicas para "escapar" a esta árvore aparente.

Falta controlo mais granular, como definir o acesso a *capabilities,* e a flexibilidade de encadear políticas.

Pex., na stack de rede esse programa pode abrir sockets, de que tipo, e as suas dependências também têm as mesmas capacidades?

AppArmor

AppArmor é um mecanismo de aplicação de MACs embebido no kernel Linux, o qual permite descrever políticas que são sempre aplicadas durante o contexto de execução. Contudo, porque as políticas podem ser mudadas e/ou desligadas em execução, é considerado um DAC nesta disciplina.

Tal como o *chroot*, permite limitar operações sobre nós na arvore de sistema. Também junta outras capacidades como permitir aplicar limites de recursos utilizados, definir se o processo tem acesso à rede e de que forma, etc.

Exemplo AppArmor

```
/home/kali/server {
        /usr/lib/x86_64-linux-gnu/libc.so.6 mr,
        /home/kali/data/** r,
        network inet udp,
}
```

Recapitulando

- Identidade em Linux: utilizadores têm um **uid** numérico e um **gid** nativo. Podem ter mais grupos associados através dos groups.
- As permissões são definidas em octal (r-w-x), na ordem de user dono do nó, grupo dono do nó, e todos os outros.
- Há bits especiais que permitem definir um **euid** que muda o utilizador efetivo durante a execução daquele nó *(pex., o executável do sudo)*
- Depois disto tudo, ainda temos mecanismos de confinamento que permitem aplicar MAC *(pex., AppArmor)*

Dúvidas?